

ỨNG DỤNG HỌC MÁY TRONG DỰ ĐOÁN NGUY CƠ UNG THƯ CỔ TỬ CUNG

Báo cáo cuối kì

Nguyễn Thị Ngọc Hân Lại Thị Huê Mẫn Thị Ngân An



Hanoi 05/2026

Nội dung

- 1 Tổng quan về bài toán
- 2 Tiền xử lý dữ liệu
- 3 Giảm chiều và trực quan hóa dữ liệu
- 4 Xây dựng mô hình
- 5 Kết quả thực nghiệm và đánh giá
- 6 Kết luận

Nội dung

- 1 Tổng quan về bài toán
- 2 Tiền xử lý dữ liệu
- 3 Giảm chiều và trực quan hóa dữ liệu
- 4 Xây dựng mô hình
- 5 Kết quả thực nghiệm và đánh giá
- 6 Kết luận

Tổng quan về bài toán

Lý do chọn đề tài

Tính cấp thiết của vấn đề: Ung thư cổ tử cung là một trong những căn bệnh phổ biến và nguy hiểm hàng đầu đối với phụ nữ toàn cầu. Việc phát hiện sớm thông qua sàng lọc các yếu tố nguy cơ có ý nghĩa quyết định trong việc nâng cao hiệu quả điều trị và giảm tỷ lệ tử vong.

Thách thức trong chẩn đoán: Các phương pháp truyền thống như Pap smear hay HPV test thường tốn kém, mất thời gian và khó tiếp cận tại các khu vực có điều kiện y tế hạn chế, đòi hỏi một phương thức hỗ trợ dự đoán nhanh chóng và hiệu quả hơn.

Học máy đã trở thành công cụ mạnh mẽ trong việc phân tích các mối quan hệ phức tạp giữa các yếu tố nguy cơ (tuổi tác, hành vi, tiền sử bệnh lý). Điều này cho phép xây dựng các mô hình dự đoán chính xác, hỗ trợ bác sĩ ra quyết định kịp thời.

Tổng quan về bài toán

Ý nghĩa và mục tiêu nghiên cứu

Đề tài sử dụng bộ dữ liệu **Cervical Cancer Risk Factors**, bao gồm các chỉ số sinh hóa quan trọng như:

- Nhóm nhân khẩu học: Age, Number of sexual partners, First sexual intercourse (age).
- Tiền sử thai sản: Num of pregnancies.
- Biện pháp tránh thai Bệnh truyền nhiễm: Hormonal Contraceptives, IUD, STDs (số lượng và thời gian sử dụng).

Mục tiêu:

- Tiền xử lý dữ liệu.
- Giảm chiều bằng PCA và LDA.
- Xây dựng mô hình KNN, Logistic Regression, SVM; đồng thời phân tích phân cụm bằng K-Means và chuyển bài toán sang hồi quy.
- Đánh giá hiệu quả các phương pháp học máy trong chẩn đoán bệnh.

Nội dung

- 1 Tổng quan về bài toán
- 2 Tiền xử lý dữ liệu
- 3 Giảm chiều và trực quan hóa dữ liệu
- 4 Xây dựng mô hình
- 5 Kết quả thực nghiệm và đánh giá
- 6 Kết luận

Tiền xử lý dữ liệu

Tổng quan về dữ liệu

Bộ dữ liệu sử dụng là **Cervical Cancer Risk Factors (UCI)**, bao gồm các thông tin sinh hóa của bệnh nhân.

Các đặc trưng chính:

- Nhóm hành vi: Age, Number of sexual partners, First sexual intercourse, Smokes.
- Nhóm bệnh lý: Num of pregnancies, STDs, STDs (number), STDs: Time since first/last diagnosis.
- Nhóm phòng ngừa: Hormonal Contraceptives, IUD (years).

Biến mục tiêu: Biopsy (0: Âm tính, 1: Dương tính)

Tiền xử lý dữ liệu

Khảo sát cấu trúc dữ liệu

Để hiểu rõ cấu trúc dữ liệu, tiến hành kiểm tra:

- Số lượng mẫu và số lượng thuộc tính: `df.shape`
- Kiểu dữ liệu của từng cột: `df.dtypes`
- Thông tin tổng quan: `df.info()`

Việc khảo sát ban đầu giúp xác định chất lượng dữ liệu trước khi huấn luyện mô hình.

- Dữ liệu tồn tại nhiều giá trị thiếu dưới dạng ký tự "?".
- Các cột liên quan đến STDs (Time since first/last diagnosis) có tỷ lệ trống cực cao (>90%).

Tiền xử lý dữ liệu

Xử lý giá trị thiếu (Missing Values)

- Thay ký tự “?” bằng giá trị NaN và chuyển dữ liệu về dạng số.
- Loại bỏ các cột có tỷ lệ thiếu quá cao: *STDs: Time since first diagnosis* và *STDs: Time since last diagnosis* do thiếu khoảng 91.72%.
- Thống kê Mean, Median, Mode và độ lệch phân phối để lựa chọn cách điền khuyết.
- Trong thực nghiệm cuối cùng, nhóm sử dụng Median cho các cột còn lại. Do nhiều biến có phân phối lệch mạnh và median ít bị ảnh hưởng bởi ngoại lai hơn Mean.

Lý do: Trung vị giúp giảm ảnh hưởng của các giá trị ngoại lai (*outliers*) tốt hơn so với việc sử dụng giá trị trung bình.

Tiền xử lý dữ liệu

Tách biến và chia tập dữ liệu

- **Tách biến đầu vào và đầu ra**

- X : Tập đặc trưng đầu vào
- y : Biến mục tiêu Biopsy

- **Chia tập dữ liệu theo quy trình hai bước**

- **Bước 1:** Tách 20% dữ liệu làm tập *Test* cố định để đánh giá khách quan mô hình cuối cùng.
- **Bước 2:** 80% dữ liệu còn lại được chia thành tập *Train* và *Validation* theo ba tỷ lệ:

$4 : 1, \quad 7 : 3, \quad 6 : 4$

- Việc chia dữ liệu được thực hiện theo phương pháp *stratified splitting* nhằm bảo toàn tỷ lệ phân bố giữa hai lớp, đặc biệt phù hợp với dữ liệu mất cân bằng.

Tiền xử lý dữ liệu

Chuẩn hóa dữ liệu và xử lý mất cân bằng

- **Chuẩn hóa dữ liệu (Standardization):**

- Đưa các đặc trưng về cùng một thang đo.
- Giúp các thuật toán dựa trên khoảng cách (KNN) hoạt động chính xác hơn.
- Công thức:

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

- Sử dụng StandardScaler từ thư viện Sklearn.

- **Xử lý mất cân bằng dữ liệu:**

- Dữ liệu có sự chênh lệch lớn giữa hai lớp.
- Áp dụng phương pháp **SMOTE** để tạo thêm dữ liệu cho lớp thiểu số (*Chỉ áp dụng trên tập huấn luyện nhằm tránh rò rỉ dữ liệu*).

Xử lý mất cân bằng dữ liệu

Thuật toán SMOTE (ý tưởng và các bước)

- Ý tưởng:
 - Thay vì sao chép nguyên xi các mẫu cũ, SMOTE tạo ra các mẫu dữ liệu **tổng hợp (synthetic)** hoàn toàn mới. Thuật toán này hoạt động trong “không gian đặc trưng” chứ không phải không gian dữ liệu gốc.
 - Nó tạo ra các điểm dữ liệu mới bằng cách **nội suy tuyến tính** giữa một điểm dữ liệu thuộc nhóm thiểu số và các “láng giềng gần nhất” của nó cũng thuộc nhóm thiểu số đó.
- Các bước thực hiện:
 - Xét một mẫu dữ liệu x_i bất kỳ thuộc lớp thiểu số.
 - Sử dụng thuật toán KNN để tìm k láng giềng gần nhất của x_i .
 - Tùy thuộc vào tỷ lệ oversampling mong muốn, thuật toán sẽ chọn ngẫu nhiên một láng giềng trong k láng giềng vừa tìm được gọi là $x_{neighbor}$.
 - Tạo mẫu dữ liệu mới dựa trên mối quan hệ giữa x_i và $x_{neighbor}$ trong không gian đặc trưng.

Xử lý mất cân bằng dữ liệu

Thuật toán SMOTE (cơ chế tạo dữ liệu)

- Mẫu dữ liệu mới được tạo bằng nội suy tuyến tính:

$$x_{new} = x_i + \lambda(x_{neighbor} - x_i), \quad \lambda \in [0, 1]$$

- Trong đó:

- x_i : Mẫu dữ liệu thuộc lớp thiểu số
- $x_{neighbor}$: Một láng giềng gần nhất của x_i
- λ : Số ngẫu nhiên trong khoảng $[0, 1]$
- x_{new} : Điểm dữ liệu tổng hợp mới được sinh ra.

Nội dung

- 1 Tổng quan về bài toán
- 2 Tiền xử lý dữ liệu
- 3 Giảm chiều và trực quan hóa dữ liệu**
- 4 Xây dựng mô hình
- 5 Kết quả thực nghiệm và đánh giá
- 6 Kết luận

Giảm chiều và trực quan hóa dữ liệu

PCA – Mục đích

PCA là phương pháp giảm chiều dữ liệu **không giám sát**, nhằm biến đổi dữ liệu ban đầu sang một không gian mới có số chiều thấp hơn nhưng vẫn giữ lại phần lớn thông tin quan trọng.

Trong bài toán này, PCA được sử dụng nhằm:

- Giảm số chiều của dữ liệu sinh hóa
- Loại bỏ sự tương quan giữa các biến
- Hỗ trợ trực quan hóa dữ liệu
- Giảm độ phức tạp tính toán

Giảm chiều và trực quan hóa dữ liệu

PCA – Cơ sở lý thuyết

Giả sử tập dữ liệu gồm N mẫu, mỗi mẫu có d đặc trưng.

- **Bước 1: Tính vectơ kỳ vọng trên toàn bộ dữ liệu**

Giúp loại bỏ phần nhiễu không cần thiết trong quá trình phân tích.

- **Bước 2: Chuẩn hóa dữ liệu**

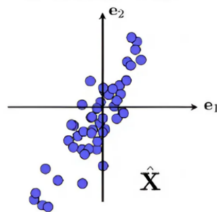
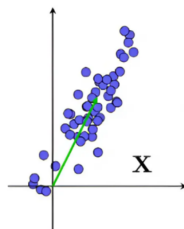
$$\hat{x}_i = x_i - \bar{x}; \quad i = 1, 2, \dots, N.$$

Việc này giúp loại bỏ sự lệch tâm của dữ liệu.

- **Bước 3: Ma trận hiệp phương sai**

$$S = \frac{1}{N} \hat{X}^T \hat{X}$$

$\hat{X}$ là ma trận dữ liệu đã được chuẩn hóa.



Giảm chiều và trực quan hóa dữ liệu

PCA – Các bước tiếp theo

- Bước 4: Phân tích trị riêng**

$$Su_i = \lambda_i u_i$$

- Bước 5: Chọn thành phần chính**

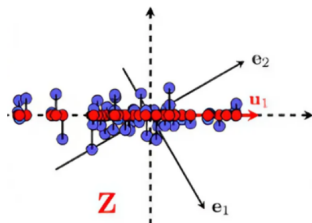
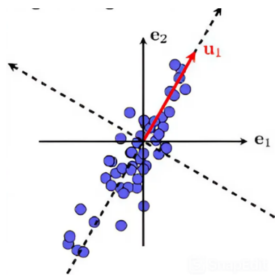
$$U_k = [u_1, u_2, \dots, u_k]$$

Các cột của $\{U_j\}_{j=1}^k$ là hệ cơ sở trực chuẩn.

- Bước 6: Chiếu dữ liệu**

$$Z = \hat{X}U_k$$

Dữ liệu đã được chuyển về không gian mới, giúp giảm chiều và tối ưu hóa quá trình tính toán.



Giảm chiều và trực quan hóa dữ liệu

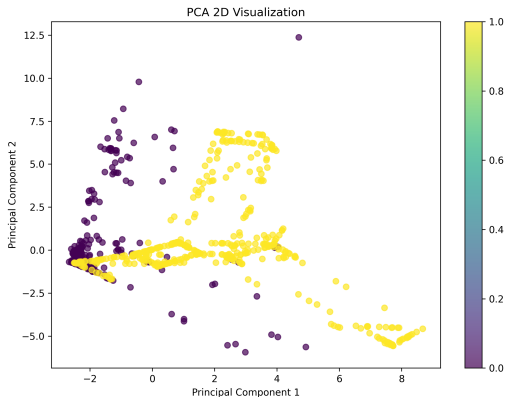
PCA – Áp dụng vào dữ liệu

Sau khi chuẩn hóa, PCA được áp dụng với mục tiêu giữ lại **95% phương sai**:
Kết quả:

- Số chiều giảm xuống còn **18 - 19** chiều tùy theo tỷ lệ chia dữ liệu.
- Điều này cho thấy dữ liệu gốc có thể tồn tại sự tương quan hoặc dư thừa thông tin giữa một số đặc trưng.
- Giảm nhiễu (noise) giúp mô hình KNN và Logistic chạy nhanh và ổn định hơn.

Giảm chiều và trực quan hóa dữ liệu

PCA – Áp dụng vào dữ liệu



Hình: Phân tán dữ liệu theo 2TPC đầu tiên

Giảm chiều và trực quan hóa dữ liệu

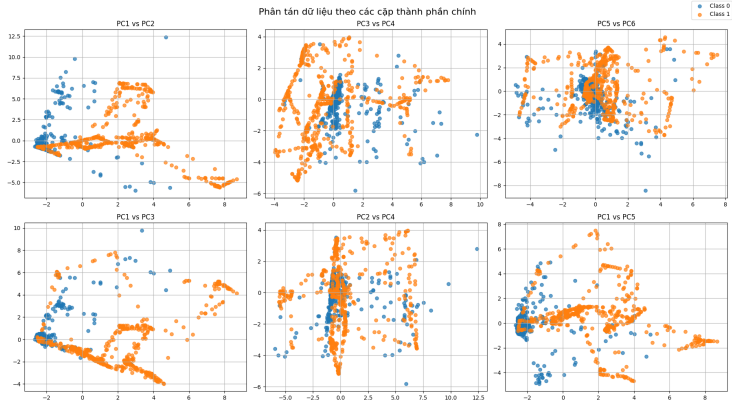
PCA – Trực quan hóa và nhận xét

Dữ liệu được chiếu xuống hai thành phần chính đầu tiên (**PC1, PC2**):

- **Cấu trúc dữ liệu:** Quan sát biểu đồ, các điểm dữ liệu màu xanh (Class 1 - Dương tính) tạo thành các dải và cụm dày đặc. Đây là kết quả của kỹ thuật SMOTE, giúp tăng cường dữ liệu lớp thiểu số để mô hình không bị lệch.
- **Hiện trạng dữ liệu:** Hai lớp dữ liệu (Đỏ - 0 và Xanh - 1) có sự chồng lấn rất lớn, đặc biệt là ở dải trung tâm và phía bên trái biểu đồ. Các yếu tố nguy cơ của ung thư cổ tử cung không tạo ra các ranh giới tuyến tính rõ ràng. Điều này minh chứng cho tính phi tuyến và mối quan hệ phức tạp giữa các biến số y sinh.
- **Kết luận:** PCA giúp giữ được cấu trúc tổng thể của dữ liệu nhưng chưa đủ để phân tách rõ rệt hai nhóm bệnh nhân.

Giảm chiều và trực quan hóa dữ liệu

PCA – Áp dụng vào dữ liệu



Hình: Phân tán dữ liệu theo 6TPC đầu tiên

Giảm chiều và trực quan hóa dữ liệu

LDA – Cơ sở lý thuyết

LDA là phương pháp giảm chiều **có giám sát**, sử dụng thông tin nhãn để tìm không gian chiều tối ưu giúp phân biệt các lớp dữ liệu.

Mục tiêu:

- Tối đa hóa khoảng cách giữa các lớp
- Tối thiểu hóa phương sai trong lớp

Với hai lớp dữ liệu:

- Trung bình lớp: m_1, m_2

Phương sai trong lớp

$$S_W = \sum (x - m_i)(x - m_i)^T$$

S_W thể hiện sự phân tán trong mỗi lớp tức là sự biến động của các điểm trong lớp so với kỳ vọng của lớp đó

Phương sai giữa lớp

$$S_B = (m_1 - m_2)(m_1 - m_2)^T$$

S_B thể hiện sự phân tán giữa các lớp, tức là khoảng cách giữa các kỳ vọng m_1 và m_2 của hai lớp

Giảm chiều và trực quan hóa dữ liệu

LDA

Sau khi tính toán được S_W và S_B , ta tính ma trận A như sau:

$$A = S_W^{-1} S_B$$

Ma trận A giúp xác định các "hướng phân biệt" dữ liệu, nhằm tối ưu hóa việc phân chia các lớp trong không gian dữ liệu.

Tính giá trị riêng của ma trận A và chọn giá trị riêng lớn nhất:

$$L = \max\{\lambda_1, \lambda_2, \dots, \lambda_d\}$$

Trong đó λ_i là các giá trị riêng của ma trận A .

Chú ý: Với mỗi giá trị riêng λ , sẽ có vô số vector riêng tương ứng.

Mục tiêu: Tìm ra các hướng có giá trị riêng lớn nhất, nhằm giảm chiều dữ liệu và tăng cường khả năng phân loại.

Giảm chiều và trực quan hóa dữ liệu

LDA – Chọn w

Sau khi có giá trị riêng lớn nhất, ta chọn vector phân biệt w sao cho:

$$(m_1 - m_2)^T w = L$$

Vector w được tính như sau:

$$w = S_W^{-1}(m_1 - m_2) \cdot \beta \quad (\text{với } \beta > 0)$$

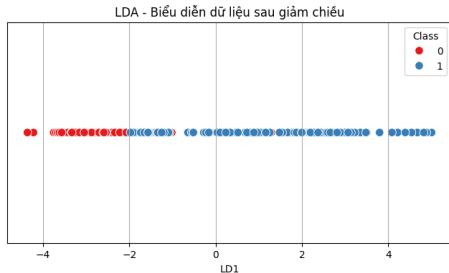
Trong đó β là hệ số điều chỉnh, giúp tối ưu hóa sự phân biệt giữa các lớp.

Giảm chiều và trực quan hóa dữ liệu

LDA – Trực quan hóa và nhận xét

Do đây là bài toán nhị phân: Dữ liệu sau LDA được giảm xuống **1 chiều**
Sau khi áp dụng LDA:

- Cải thiện rõ rệt so với PCA; ranh giới phân loại xuất hiện rõ ràng quanh giá trị -2 .
- Khoảng cách giữa nhóm bệnh nhân bình thường và nhóm nguy cơ được tối đa hóa.
- Giảm thiểu chồng lấn dữ liệu, tối ưu hóa không gian cho các thuật toán phân loại.



Nội dung

- 1 Tổng quan về bài toán
- 2 Tiền xử lý dữ liệu
- 3 Giảm chiều và trực quan hóa dữ liệu
- 4 Xây dựng mô hình**
- 5 Kết quả thực nghiệm và đánh giá
- 6 Kết luận

Xây dựng mô hình

K-Nearest Neighbors (KNN) – Giới thiệu

KNN là thuật toán học có giám sát đơn giản nhưng hiệu quả, được sử dụng phổ biến trong các bài toán phân loại.

Điểm đặc biệt:

- Không xây dựng mô hình tường minh trong giai đoạn huấn luyện
- Lưu trữ toàn bộ dữ liệu huấn luyện
- Dự đoán dựa trên các điểm lân cận gần nhất

Xây dựng mô hình

KNN – Nguyên lý hoạt động

Để phân loại một điểm dữ liệu mới x , thuật toán thực hiện:

- 1 Tính khoảng cách từ x đến tất cả các điểm trong tập huấn luyện
- 2 Chọn ra K điểm gần nhất
- 3 Xác định nhãn của x dựa trên đa số nhãn của K điểm đó

Khoảng cách này được tính xác định như sau:

- **Trường hợp $d = 1$ (dữ liệu một chiều):**

Khoảng cách giữa điểm cần dự đoán x và từng điểm huấn luyện x_i được tính theo công thức:

$$d_i = |x - x_i|$$

- **Trường hợp $d > 1$ (dữ liệu nhiều chiều):**

Khi dữ liệu là vector nhiều chiều, khoảng cách giữa x và x_i được tính bằng công thức chuẩn Euclid (L2 norm):

$$d_i = \|x - x_i\|_2 = \sqrt{\sum_{j=1}^d (x_j - x_{i,j})^2}$$

Xây dựng mô hình

KNN – Lựa chọn tham số K

Tham số K ảnh hưởng trực tiếp đến hiệu suất mô hình:

Nguyên lý

- Phân loại dựa trên khoảng cách giữa các điểm dữ liệu láng giềng.

Tối ưu hóa

- Sử dụng GridSearchCV để khảo sát giá trị K trong khoảng $[3, 21]$.

Xây dựng mô hình

Logistic Regression – Giới thiệu

Hồi quy logistic mô hình hóa **xác suất có điều kiện** (conditional probability) của một điểm dữ liệu thuộc về lớp dương (thường là lớp 1). Cụ thể, mô hình ước lượng xác suất $P(y = 1|x; \theta)$, trong đó:

- x là vector các đặc trưng đầu vào ($x \in \mathbb{R}^{n+1}$, với $x_0 = 1$ là hệ số chặn).
- θ là vector các tham số (trọng số) của mô hình ($\theta \in \mathbb{R}^{n+1}$).

Giá trị đầu ra của mô hình, ký hiệu là $h_\theta(x)$, biểu diễn xác suất này:

$$h_\theta(x) = P(y = 1|x; \theta)$$

Xây dựng mô hình

Logistic Regression – Mô hình toán học

Để đảm bảo rằng giá trị đầu ra $h_{\theta}(x)$ nằm trong khoảng $(0, 1)$, hồi quy logistic sử dụng **hàm sigmoid** (còn gọi là hàm logistic) để biến đổi tổ hợp tuyến tính của các đặc trưng và tham số ($\theta^T x$). Hàm sigmoid được định nghĩa là:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Hàm này nhận đầu vào là một số thực bất kỳ ($z \in \mathbb{R}$) và trả về một giá trị trong khoảng $(0, 1)$. Do đó, giả thuyết (hypothesis) của mô hình hồi quy logistic được viết là:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Trong đó:

- θ : vector trọng số
- x : vector đặc trưng

Giá trị $h_{\theta}(x)$ biểu diễn xác suất thuộc lớp 1.

Xây dựng mô hình

Logistic Regression – Quy tắc phân loại

Sau khi tính được xác suất $h_{\theta}(x)$, ta chuyển đổi nó thành dự đoán lớp cụ thể bằng quy tắc ngưỡng (thường là 0.5):

$$\hat{y} = \begin{cases} 1 & \text{nếu } h_{\theta}(x) \geq 0.5 \\ 0 & \text{nếu } h_{\theta}(x) < 0.5 \end{cases}$$

Điều này tương đương với việc kiểm tra $\theta^T x \geq 0$, vì hàm sigmoid $g(z) \geq 0.5$ khi và chỉ khi $z \geq 0$.

Xây dựng mô hình

Logistic Regression – Hàm mất mát

Để huấn luyện mô hình, ta cần một hàm mất mát đánh giá sai lệch giữa dự đoán và giá trị thực. Hồi quy logistic sử dụng Log Loss (hay Binary Cross-Entropy):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

Trong đó:

- m : số lượng mẫu huấn luyện.
- $y^{(i)}$: nhãn thực tế của mẫu thứ i .
- $h_{\theta}(x_i)$: xác suất dự đoán cho mẫu thứ i .

Xây dựng mô hình

Logistic Regression – Regularization

Để tránh hiện tượng **overfitting**, ta thêm một thành phần phạt vào hàm mất mát (Cross-Entropy):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

- λ : hệ số điều chỉnh mức độ regularization.
- Khi λ lớn: mô hình đơn giản hơn (tránh overfit).
- Khi λ nhỏ: mô hình linh hoạt hơn.

Ý nghĩa: Hạn chế giá trị lớn của các tham số θ_j (với $j \geq 1$), giúp đường phân chia mượt mà và tổng quát tốt hơn trên dữ liệu mới.

Xây dựng mô hình

Logistic Regression – Tối ưu hóa mô hình

Mục tiêu là tìm θ sao cho $J(\theta)$ nhỏ nhất. Phương pháp phổ biến là **Gradient Descent**:

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

với α là tốc độ học (learning rate). Đạo hàm của hàm mất mát (có thêm thành phần phạt L2) được tính bằng 1 công thức tổng quát:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} \theta_j$$

Xây dựng mô hình

Support Vector Machine (SVM) – Giới thiệu

Support Vector Machine (SVM) là thuật toán học máy có giám sát, thường được sử dụng trong các bài toán phân loại.

Ý tưởng chính của SVM là tìm một siêu phẳng phân tách tối ưu giữa các lớp dữ liệu sao cho khoảng cách từ siêu phẳng đó đến các điểm dữ liệu gần nhất của mỗi lớp là lớn nhất.

Trong bài toán này:

- $y = 0$: Không có dấu hiệu ung thư cổ tử cung.
- $y = 1$: Có dấu hiệu ung thư cổ tử cung.

Xây dựng mô hình

SVM – Siêu phẳng phân tách

Giả sử dữ liệu đầu vào là vector đặc trưng:

$$x = (x_1, x_2, \dots, x_d)$$

với d là số chiều dữ liệu, SVM tìm một siêu phẳng có dạng:

$$w^T x + b = 0$$

Trong đó:

- w : vector trọng số.
- b : hệ số bias.
- x : vector đặc trưng đầu vào.

Quy tắc phân loại:

$$\hat{y} = \begin{cases} 1, & \text{nếu } w^T x + b \geq 0 \\ 0, & \text{nếu } w^T x + b < 0 \end{cases}$$

Xây dựng mô hình

SVM – Margin

SVM không chọn một siêu phẳng bất kỳ, mà tìm siêu phẳng sao cho khoảng cách giữa hai lớp dữ liệu là lớn nhất. Khoảng cách này được gọi là *margin*.

Hai siêu phẳng support tương ứng:

$$w^T x + b = 1$$

$$w^T x + b = -1$$

Khoảng cách từ siêu phẳng phân tách đến điểm dữ liệu gần nhất được gọi là *margin* và được xác định bởi:

$$\text{margin} = \min_n \frac{y_n(w^T x_n + b)}{\|w\|_2}.$$

Xây dựng mô hình

SVM – Bài toán tối ưu

Bài toán tối ưu của SVM được đưa về bài toán cực tiểu hóa:

$$\min_{w,b} \frac{1}{2} \|w\|_2^2$$

với ràng buộc:

$$y_i(w^T x_i + b) \geq 1$$

Trong đó:

- x_i là vector đặc trưng của mẫu thứ i .
- $y_i \in \{-1, 1\}$ là nhãn lớp tương ứng.

Trong lý thuyết SVM, nhãn thường được mã hóa dưới dạng $y_i \in \{-1, 1\}$. Tuy nhiên, trong bộ dữ liệu Biopsy, nhãn ban đầu có dạng 0 và 1, nên khi huấn luyện mô hình có thể ánh xạ $0 \rightarrow -1, 1 \rightarrow 1$, sau đó đổi kết quả dự đoán về dạng 0/1 để diễn giải.

Xây dựng mô hình

SVM – Soft Margin

Dữ liệu thực tế thường chứa nhiễu, chồng lấn hoặc không thể phân loại tuyến tính. Để khắc phục, Soft Margin SVM được sử dụng bằng cách cho phép một số điểm dữ liệu nằm trong vùng margin hoặc bị phân loại sai thông qua việc đưa biến slack ξ_i vào mô hình.

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Bài toán tối ưu của Soft Margin SVM trở thành:

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 + C \sum_{n=1}^N \xi_n$$

với điều kiện ràng buộc:

$$\begin{aligned} 1 - \xi_n - y_n(w^T x_n + b) &\leq 0 \\ -\xi_n &\leq 0, \forall n = 1, 2, \dots, N. \end{aligned}$$

- $\|w\|_2^2$ giúp cực đại hóa margin.
- $\sum_{i=1}^n \xi_i$ là tổng mức độ vi phạm margin.
- C là tham số điều chỉnh mức độ phạt đối với các điểm vi phạm.

Xây dựng mô hình

SVM – Ý nghĩa tham số C

- **C lớn:**

- Mô hình ưu tiên phân loại đúng dữ liệu huấn luyện.
- Ranh giới có thể phức tạp hơn.
- Nguy cơ overfitting cao hơn.

- **C nhỏ:**

- Cho phép nhiều điểm nằm trong margin hơn.
- Ranh giới mượt hơn.
- Khả năng tổng quát hóa có thể tốt hơn.

Trong đề tài này, Soft Margin SVM được sử dụng do dữ liệu thực tế có thể chứa nhiễu và không phân tách tuyến tính hoàn toàn.

Xây dựng mô hình

SVM – Kernel SVM

Khi dữ liệu không phân tách tốt bằng một siêu phẳng tuyến tính trong không gian ban đầu, SVM sử dụng **kernel** để ánh xạ dữ liệu sang không gian đặc trưng mới có số chiều cao hơn.

Một số kernel phổ biến:

- **Kernel tuyến tính:**

$$K(x_i, x_j) = x_i^T x_j$$

- **Kernel đa thức:**

$$K(x_i, x_j) = (x_i^T x_j + c)^d$$

- **Kernel Gaussian RBF:**

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Trong thực nghiệm, nhóm khảo sát các kernel tuyến tính và RBF bằng GridSearchCV. Việc lựa chọn kernel và các tham số C, gamma được thực hiện dựa trên kết quả đánh giá Recall trên tập huấn luyện/cross-validation.

Xây dựng mô hình

K-Means – Giới thiệu

K-Means là một trong những thuật toán phân cụm phổ biến nhất trong học máy không giám sát (*Unsupervised Learning*). Mục tiêu của thuật toán là phân chia tập dữ liệu thành K cụm dữ liệu sao cho các điểm dữ liệu trong cùng một cụm có mức độ tương đồng cao, trong khi các điểm thuộc các cụm khác nhau có mức độ khác biệt lớn hơn.

Trong đề tài, K-Means được sử dụng nhằm:

- Phân tích cấu trúc tiềm ẩn của dữ liệu.
- Quan sát khả năng hình thành các nhóm dữ liệu tự nhiên.
- Đánh giá mối liên hệ giữa các cụm dữ liệu với nhãn thực tế.
- Hỗ trợ trực quan hóa dữ liệu sau khi giảm chiều.

Xây dựng mô hình

K-Means – Biểu diễn dữ liệu

Giả sử tập dữ liệu gồm:

$$X = \{x_n\}_{n=1}^N$$

trong đó, $x_n \in \mathbb{R}^d$ là vector đặc trưng của mẫu dữ liệu thứ n .

Thuật toán cần phân chia dữ liệu thành K cụm với $1 < K < N$, mỗi cụm được đại diện bởi một tâm cụm:

$$M = \{m_k\}_{k=1}^K$$

Trong đó m_k là tâm cụm thứ k .

Xây dựng mô hình

K-Means – Biến ẩn gán cụm

Để biểu diễn việc một điểm dữ liệu thuộc về cụm nào, K-Means sử dụng biến ẩn:

$$y_{n,j} \in \{0, 1\}$$

- $y_{n,j} = 1$ nếu điểm dữ liệu x_n thuộc cụm thứ j .
- $y_{n,j} = 0$ trong trường hợp ngược lại.

Mỗi điểm dữ liệu chỉ thuộc đúng một cụm, do đó:

$$\sum_{j=1}^K y_{n,j} = 1$$

Biến $y_{n,j}$ được xem như một *latent variable* giúp mô tả cấu trúc cụm của dữ liệu.

Xây dựng mô hình

K-Means – Hàm mục tiêu

K-Means thực chất là bài toán tối thiểu hóa **tổng khoảng cách Euclidean bình phương** từ các điểm dữ liệu đến tâm cụm tương ứng.

Giả sử các điểm dữ liệu được phân vào cụm thứ k , tổng sai số nội cụm được xác định bởi:

$$\sum_{x_n \in \text{Cluster}_k} \|x_n - m_k\|_2^2$$

Tổng sai số trên toàn bộ các cụm:

$$L(Y, M) = \sum_{n=1}^N \sum_{j=1}^K y_{n,j} \|x_n - m_j\|_2^2$$

Trong đó:

- Y : tập các biến ẩn $y_{n,j}$.
- M : tập các tâm cụm.
- $\|x_n - m_j\|_2^2$: khoảng cách Euclidean bình phương giữa điểm dữ liệu và tâm cụm.

Xây dựng mô hình

K-Means – Bài toán tối ưu

Mục tiêu của thuật toán là tìm cách gán cụm Y và tập tâm cụm M sao cho hàm mất mát nhỏ nhất:

$$(Y^*, M^*) = \arg \min_{Y, M} L(Y, M)$$

với các ràng buộc:

$$y_{n,j} \in \{0, 1\}$$

và:

$$\sum_{j=1}^K y_{n,j} = 1$$

Việc tối thiểu hóa hàm mất mát trên giúp các điểm dữ liệu trong cùng một cụm nằm gần nhau hơn.

Xây dựng mô hình

K-Means – Hàm mục tiêu

K-Means thực chất là bài toán tối thiểu hóa **tổng phương sai nội bộ** của các cụm dữ liệu.

Hàm mục tiêu:

$$L(Y, M) = \sum_{n=1}^N \sum_{j=1}^K y_{n,j} \|x_n - m_j\|_2^2$$

Trong đó:

- Y : tập các biến ẩn $y_{n,j}$.
- M : tập các tâm cụm.
- $\|x_n - m_j\|_2^2$: khoảng cách Euclid bình phương từ điểm dữ liệu đến tâm cụm.

Mục tiêu của thuật toán:

$$(Y^*, M^*) = \arg \min_{Y, M} L(Y, M)$$

Xây dựng mô hình

K-Means – Khoảng cách Euclidean

Trong K-Means, khoảng cách giữa các điểm dữ liệu thường được tính bằng khoảng cách Euclidean.

Với:

$$x_i = (x_{i1}, x_{i2}, \dots, x_{id})$$

và

$$x_j = (x_{j1}, x_{j2}, \dots, x_{jd})$$

Khoảng cách Euclidean được xác định bởi:

$$d(x_i, x_j) = \sqrt{\sum_{m=1}^d (x_{im} - x_{jm})^2}$$

Ý nghĩa:

- Khoảng cách càng nhỏ thì hai điểm dữ liệu càng giống nhau.
- Các điểm gần nhau thường có xu hướng thuộc cùng một cụm.

Xây dựng mô hình

K-Means – Quy tắc gán cụm

Giả sử tại một bước lặp đã biết các tâm cụm: $\{m_c\}_{c=1}^K$. Mỗi điểm dữ liệu x_n sẽ được gán vào cụm có tâm gần nhất. Cụ thể, $y_{n,j} = 1$ khi và chỉ khi:

$$\|x_n - m_j\|_2^2 = \min_c \|x_n - m_c\|_2^2$$

Ngược lại:

$$y_{n,j} = 0$$

Nhận xét:

- Mỗi điểm được gán vào đúng một cụm.
- Cụm được chọn là cụm có tâm gần điểm đó nhất.

Xây dựng mô hình

K-Means – Cập nhật tâm cụm

Sau khi hoàn thành bước gán cụm, tâm cụm được cập nhật bằng trung bình cộng của các điểm dữ liệu thuộc cụm đó.

Tâm cụm thứ k được tính theo công thức:

$$m_k = \frac{\sum_{n=1}^N y_{n,k} x_n}{\sum_{n=1}^N y_{n,k}}$$

Trong đó:

- Tử số là tổng các điểm dữ liệu thuộc cụm thứ k .
- Mẫu số là số lượng phần tử trong cụm.

Việc cập nhật này giúp tâm cụm tiến gần hơn tới vùng có mật độ dữ liệu cao.

Xây dựng mô hình

K-Means – Các bước thực hiện

Thuật toán K-Means được thực hiện theo quy trình lặp:

- ➊ **Khởi tạo tâm cụm:** Chọn ngẫu nhiên K điểm dữ liệu làm tâm cụm ban đầu.
- ➋ **Gán cụm cho dữ liệu:** Với mỗi điểm x_n , tính khoảng cách tới các tâm cụm và gán vào cụm có tâm gần nhất.
- ➌ **Cập nhật tâm cụm:** Tính lại tâm cụm bằng trung bình cộng của các điểm trong cụm.
- ➍ **Kiểm tra điều kiện dừng:** Lặp lại hai bước trên cho đến khi tâm cụm không thay đổi đáng kể hoặc đạt số vòng lặp tối đa.

Xây dựng mô hình

K-Means – Lựa chọn số cụm bằng Elbow Method

Trong thuật toán K-Means, số cụm K cần được xác định trước khi huấn luyện mô hình. Việc lựa chọn K phù hợp ảnh hưởng trực tiếp đến chất lượng phân cụm.

- Nếu K quá nhỏ, các cụm khác nhau có thể bị gộp chung.
- Nếu K quá lớn, dữ liệu có thể bị chia thành nhiều cụm nhỏ và khó diễn giải.

Để lựa chọn số cụm phù hợp, nhóm sử dụng Elbow Method. Phương pháp này dựa trên giá trị WCSS và được tính theo công thức:

$$WCSS = \sum_{i=1}^K \sum_{x_j \in C_i} \|x_j - \mu_i\|_2^2$$

Trong đó:

- K : số cụm.
- C_i : cụm thứ i .
- x_j : điểm dữ liệu thuộc cụm C_i .
- μ_i : tâm của cụm C_i .

Xây dựng mô hình

K-Means – Đánh giá bằng Silhouette Score

Khi tăng số cụm K , WCSS giảm dần. Giá trị K tại điểm đường cong bắt đầu giảm chậm, tạo dạng “khuỷu tay”, được xem là số cụm phù hợp.

Bên cạnh Elbow Method, nhóm sử dụng thêm Silhouette Score để đánh giá chất lượng phân cụm:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Trong đó:

- $a(i)$: khoảng cách trung bình từ điểm x_i đến các điểm khác trong cùng cụm.
- $b(i)$: khoảng cách trung bình nhỏ nhất từ điểm x_i đến các điểm thuộc cụm khác gần nhất.

Giá trị Silhouette Score nằm trong khoảng $[-1, 1]$:

- Gần 1: cụm được phân tách tốt.
- Gần 0: các cụm có sự chồng lấn.
- Âm: điểm dữ liệu có thể bị gán sai cụm.

Nội dung

- 1 Tổng quan về bài toán
- 2 Tiền xử lý dữ liệu
- 3 Giảm chiều và trực quan hóa dữ liệu
- 4 Xây dựng mô hình
- 5 Kết quả thực nghiệm và đánh giá**
- 6 Kết luận

Kết quả thực nghiệm và đánh giá

Thiết lập thực nghiệm

Sau tiền xử lý, dữ liệu gồm:

- **858 mẫu**
- **33 đặc trưng đầu vào**
- Biến mục tiêu Biopsy

Phân bố lớp:

- Lớp 0: **803 mẫu**
- Lớp 1: **55 mẫu**

Nhận xét: Dữ liệu mất cân bằng mạnh, do đó cần áp dụng SMOTE trên tập huấn luyện.

Kết quả thực nghiệm và đánh giá

Quy trình chia dữ liệu

Nhóm thực hiện đánh giá theo hai bước:

- **Bước 1:** Tách riêng **20% dữ liệu** làm tập *Test cố định*.
- **Bước 2:** Phần dữ liệu còn lại được chia thành *Train* và *Validation* theo ba tỷ lệ 4:1, 7:3 và 6:4.

Sau bước tách *Test cố định*, dữ liệu được chia thành:

Tập dữ liệu	Số mẫu	Lớp 0	Lớp 1
Toàn bộ dữ liệu	858	803	55
X_{remain}	686	642	44
Test cố định	172	161	11

Kết quả thực nghiệm và đánh giá

Các mô hình và chỉ số đánh giá

Các mô hình sử dụng:

- K-Nearest Neighbors (KNN)
- Logistic Regression
- Support Vector Machine (SVM)

Các không gian dữ liệu:

- Dữ liệu gốc sau chuẩn hóa
- Dữ liệu sau PCA
- Dữ liệu sau LDA

Các chỉ số đánh giá gồm: *Accuracy, Precision, Recall, F1-score* và *Confusion Matrix*

Kết quả thực nghiệm và đánh giá

Tỷ lệ Train:Validation = 4:1

Với tỷ lệ Train:Validation = 4:1:

- Train: **548 mẫu**
- Validation: **138 mẫu**
- Sau SMOTE, tập Train cân bằng thành: **[513, 513]**
- PCA giảm dữ liệu xuống **19 chiều**
- Phương sai giữ lại: khoảng **96.00%**

Kết quả thực nghiệm và đánh giá

Kết quả trên tập Validation – Tỷ lệ 4:1

Mô hình	Dữ liệu	Accuracy	Precision	Recall	F1-score
KNN	Original	0.9638	0.7500	0.6667	0.7059
	PCA	0.9638	0.7500	0.6667	0.7059
	LDA	0.9710	0.7273	0.8889	0.8000
Logistic	Original	0.9638	0.7000	0.7778	0.7368
	PCA	0.9783	0.8000	0.8889	0.8421
	LDA	0.9783	0.8000	0.8889	0.8421
SVM	Original	0.9348	0.5000	0.2222	0.3077
	PCA	0.9275	0.4000	0.2222	0.2857
	LDA	0.9783	0.8000	0.8889	0.8421

Kết quả thực nghiệm và đánh giá

Kết quả trên Test cố định – Tỷ lệ 4:1

Mô hình	Dữ liệu	Accuracy	Precision	Recall	F1-score
KNN	Original	0.9477	0.5714	0.7273	0.6400
	PCA	0.9535	0.6154	0.7273	0.6667
	LDA	0.9419	0.5294	0.8182	0.6429
Logistic	Original	0.9651	0.6923	0.8182	0.7500
	PCA	0.9651	0.6923	0.8182	0.7500
	LDA	0.9593	0.6429	0.8182	0.7200
SVM	Original	0.9477	0.5833	0.6364	0.6087
	PCA	0.9477	0.6000	0.5455	0.5714
	LDA	0.9593	0.6429	0.8182	0.7200

Kết quả thực nghiệm và đánh giá

Nhận xét kết quả trên Test cố định – Tỷ lệ 4:1

- **Logistic Regression** trên dữ liệu gốc và dữ liệu sau giảm chiều bằng PCA cho kết quả tốt nhất:
 - Accuracy = 0.9651
 - Precision = 0.6923
 - Recall = 0.8182
 - F1-score = 0.7500
- Mô hình phát hiện đúng **9 trên 11 mẫu thuộc lớp 1** trong tập *Test* cố định.
- **KNN** đạt kết quả tốt nhất khi sử dụng PCA với Accuracy = 0.9535 và F1-score = 0.6667, tuy nhiên Recall thấp hơn Logistic Regression.
- **SVM** được cải thiện rõ rệt khi kết hợp với LDA, đạt Recall = 0.8182 và F1-score = 0.7200.
- Nhìn chung, với tỷ lệ 4:1, **Logistic Regression** trên dữ liệu gốc và PCA là lựa chọn phù hợp hơn do có Recall cao và F1-score tốt.

Kết quả thực nghiệm và đánh giá

Tỷ lệ Train:Validation = 7:3

Với tỷ lệ 7:3:

- Train: **480 mẫu**
- Validation: **206 mẫu**
- Sau SMOTE, tập Train cân bằng thành: **[449, 449]**
- PCA giảm dữ liệu xuống **18 chiều**
- Phương sai giữ lại: khoảng **95.12%**

Kết quả thực nghiệm và đánh giá

Kết quả trên tập Validation – Tỷ lệ 7:3

Mô hình	Dữ liệu	Accuracy	Precision	Recall	F1-score
KNN	Original	0.9369	0.5000	0.4615	0.4800
	PCA	0.9320	0.4667	0.5385	0.5000
	LDA	0.9757	0.7500	0.9231	0.8276
Logistic	Original	0.9709	0.7059	0.9231	0.8000
	PCA	0.9612	0.6471	0.8462	0.7333
	LDA	0.9757	0.7500	0.9231	0.8276
SVM	Original	0.9369	0.5000	0.1538	0.2353
	PCA	0.9417	0.5556	0.3846	0.4545
	LDA	0.9757	0.7500	0.9231	0.8276

Kết quả thực nghiệm và đánh giá

Kết quả trên Test cố định – Tỷ lệ 7:3

Mô hình	Dữ liệu	Accuracy	Precision	Recall	F1-score
KNN	Original	0.9419	0.5385	0.6364	0.5833
	PCA	0.9186	0.4000	0.5455	0.4615
	LDA	0.9593	0.6429	0.8182	0.7200
Logistic	Original	0.9593	0.6429	0.8182	0.7200
	PCA	0.9593	0.6429	0.8182	0.7200
	LDA	0.9593	0.6429	0.8182	0.7200
SVM	Original	0.9419	0.5455	0.5455	0.5455
	PCA	0.9302	0.4615	0.5455	0.5000
	LDA	0.9593	0.6429	0.8182	0.7200

Kết quả thực nghiệm và đánh giá

Nhận xét kết quả – Tỷ lệ 7:3

- Trên tập *Validation*, các mô hình sử dụng **LDA** đạt kết quả cao nhất:

$$Acc = 0.9757, \quad Precision = 0.7500, \quad Recall = 0.9231, \quad F1\text{-score} = 0.8276$$

Cho thấy LDA giúp cải thiện khả năng phân tách hai lớp, đặc biệt hiệu quả với **SVM**.

- Trên tập *Test* cố định, các mô hình tốt nhất gồm **KNN-LDA**, **Logistic Regression** và **SVM-LDA** đều đạt:

$$Acc = 0.9593, \quad Precision = 0.6429, \quad Recall = 0.8182, \quad F1\text{-score} = 0.7200$$

- Kết quả này tương ứng với việc mô hình phát hiện đúng 9/11 mẫu lớp 1 trong tập *Test* cố định.
- Nhìn chung, với tỷ lệ 7:3, **Logistic Regression** vẫn là mô hình ổn định hơn khi xét khả năng phát hiện lớp 1 và khả năng tổng quát hóa trên tập kiểm tra cố định.

Kết quả thực nghiệm và đánh giá

Tỷ lệ Train:Validation = 6:4

Với tỷ lệ 6:4:

- Train: **411 mẫu**
- Validation: **275 mẫu**
- Phân bố tập Train ban đầu: **[385, 26]**
- Sau SMOTE, tập Train cân bằng thành: **[385, 385]**
- Phân bố tập Validation: **[257, 18]**
- PCA giảm dữ liệu xuống **18 chiều**
- Phương sai giữ lại: khoảng **95.13%**

Kết quả thực nghiệm và đánh giá

Kết quả trên tập Validation – Tỷ lệ 6:4

Mô hình	Dữ liệu	Accuracy	Precision	Recall	F1-score
KNN	Original	0.9382	0.5294	0.5000	0.5143
	PCA	0.9273	0.4545	0.5556	0.5000
	LDA	0.9636	0.7000	0.7778	0.7368
Logistic	Original	0.9527	0.5926	0.8889	0.7111
	PCA	0.9600	0.6842	0.7222	0.7027
	LDA	0.9673	0.6957	0.8889	0.7805
SVM	Original	0.9491	0.6667	0.4444	0.5333
	PCA	0.9455	0.6000	0.5000	0.5455
	LDA	0.9636	0.6667	0.8889	0.7619

Kết quả thực nghiệm và đánh giá

Kết quả trên Test cố định – Tỷ lệ 6:4

Mô hình	Dữ liệu	Accuracy	Precision	Recall	F1-score
KNN	Original	0.9302	0.4545	0.4545	0.4545
	PCA	0.9244	0.4167	0.4545	0.4348
	LDA	0.9593	0.6429	0.8182	0.7200
Logistic	Original	0.9593	0.6250	0.9091	0.7407
	PCA	0.9419	0.5385	0.6364	0.5833
	LDA	0.9651	0.6923	0.8182	0.7500
SVM	Original	0.9360	0.5000	0.4545	0.4762
	PCA	0.9302	0.4545	0.4545	0.4545
	LDA	0.9593	0.6429	0.8182	0.7200

Kết quả thực nghiệm và đánh giá

Nhận xét kết quả – Tỷ lệ 6:4

- Với tỷ lệ 6:4, **Logistic Regression trên dữ liệu gốc** cho kết quả nổi bật nhất nếu xét theo *Recall* trên tập *Test* cố định.
- Mô hình đạt $Recall = 0.9091$, tương ứng với việc phát hiện đúng **10/11 mẫu lớp 1**. Đây là kết quả quan trọng trong bài toán dự đoán nguy cơ ung thư cổ tử cung.
- Khi sử dụng **PCA**, hiệu quả của một số mô hình không được cải thiện rõ rệt. Đặc biệt, Logistic Regression trên dữ liệu PCA chỉ đạt $Recall = 0.6364$ trên tập *Test*.
- Ngược lại, **LDA** giúp cải thiện rõ đối với KNN và SVM. Cụ thể, KNN-LDA và SVM-LDA đều đạt $Recall = 0.8182$ trên tập *Test* cố định.
- Nhìn chung, **Logistic Regression trên dữ liệu gốc** phù hợp nhất nếu ưu tiên *Recall*, trong khi Logistic Regression-LDA cho kết quả cân bằng hơn giữa Accuracy, Precision, Recall và F1-score.

Kết quả thực nghiệm và đánh giá

So sánh tổng hợp các mô hình trên tập Test cố định

Tỷ lệ	Mô hình	Dữ liệu	Acc	Prec	Recall	F1-score
4:1	Logistic	Original	0.9651	0.6923	0.8182	0.7500
4:1	Logistic	PCA	0.9651	0.6923	0.8182	0.7500
7:3	Logistic	Original/PCA/LDA	0.9593	0.6429	0.8182	0.7200
7:3	KNN/SVM	LDA	0.9593	0.6429	0.8182	0.7200
6:4	Logistic	Original	0.9593	0.6250	0.9091	0.7407
6:4	Logistic	LDA	0.9651	0.6923	0.8182	0.7500

Kết quả thực nghiệm và đánh giá

Nhận xét so sánh tổng hợp trên Test cố định

- **Logistic Regression** là mô hình cho kết quả ổn định nhất trên tập *Test* cố định.
- Ở tỷ lệ 4:1, Logistic Regression trên dữ liệu gốc và PCA đều đạt kết quả tốt:

$$Acc = 0.9651, \quad Precision = 0.6923, \quad Recall = 0.8182, \quad F1 = 0.7500$$

- Ở tỷ lệ 6:4, Logistic Regression kết hợp với LDA cũng đạt $F1\text{-score} = 0.7500$, cho thấy mô hình có sự cân bằng tốt giữa các chỉ số đánh giá.
- Đáng chú ý, Logistic Regression trên dữ liệu gốc với tỷ lệ 6:4 đạt *Recall* cao nhất:

$$Recall = 0.9091$$

tương ứng với việc phát hiện đúng 10/11 mẫu lớp 1 trong tập *Test* cố định.

- Do bài toán phát hiện nguy cơ ung thư cổ tử cung ưu tiên giảm số ca bệnh bị bỏ sót, **Logistic Regression** trên dữ liệu gốc với tỷ lệ 6:4 được xem là lựa chọn phù hợp nhất.

Kết quả thực nghiệm và đánh giá

Đánh giá hiện tượng overfitting

- Để kiểm tra overfitting, nhóm so sánh kết quả giữa tập *Train* và tập *Validation*.
- Nếu mô hình đạt kết quả rất cao trên *Train* nhưng giảm mạnh trên *Validation*, mô hình có dấu hiệu học quá khớp dữ liệu huấn luyện.
- Kết quả thực nghiệm cho thấy **KNN** và **SVM** có dấu hiệu overfitting rõ hơn:
 - KNN trên dữ liệu gốc, tỷ lệ 7:3:

$$Recall_{Train} = 1.0000, \quad Recall_{Val} = 0.4615$$

- SVM trên dữ liệu gốc, tỷ lệ 7:3:

$$Recall_{Train} = 1.0000, \quad Recall_{Val} = 0.1538$$

- Ngược lại, **Logistic Regression** ổn định hơn:

$$Recall_{Train} = 0.9621, \quad Recall_{Val} = 0.9231$$

- Nhìn chung, Logistic Regression có khả năng tổng quát hóa tốt hơn và phù hợp hơn khi xét đến khả năng tránh overfitting.

Kết quả thực nghiệm và đánh giá

Đánh giá tác động của PCA

- **PCA** giúp giảm số chiều dữ liệu từ **33 chiều** ban đầu xuống còn khoảng **18–19 chiều**.
- Đồng thời, PCA vẫn giữ lại trên **95%** phương sai của dữ liệu:
 - Tỷ lệ 4:1: giảm xuống **19 chiều**, giữ lại khoảng **96.00%** phương sai.
 - Tỷ lệ 7:3 và 6:4: PCA giảm xuống 18 chiều, giữ lại khoảng 95.1% phương sai.
- Tuy nhiên, PCA không phải lúc nào cũng cải thiện kết quả phân loại. Ví dụ, với tỷ lệ 6:4 trên tập *Test* cố định:

$$Recall_{\text{Logistic-PCA}} = 0.6364 < Recall_{\text{Logistic-Original}} = 0.9091$$

Điều này cho thấy PCA giúp giảm chiều và giảm độ phức tạp tính toán, do PCA không sử dụng thông tin nhãn lớp, nên các thành phần chính được giữ lại chưa chắc là các hướng phân biệt tốt nhất giữa hai lớp.

Kết quả thực nghiệm và đánh giá

Đánh giá tác động của LDA

- **LDA** thường giúp cải thiện khả năng nhận diện lớp 1, đặc biệt đối với **KNN** và **SVM**.
- Với tỷ lệ 7:3 trên tập *Test* cố định, KNN được cải thiện rõ rệt khi dùng LDA:

$$Recall_{KNN-Original} = 0.6364 \rightarrow Recall_{KNN-LDA} = 0.8182$$

- Tương tự, SVM cũng được cải thiện khi kết hợp với LDA:

$$Recall_{SVM-Original} = 0.5455 \rightarrow Recall_{SVM-LDA} = 0.8182$$

- Khác với PCA, **LDA là phương pháp giảm chiều có giám sát**, tức là có sử dụng thông tin nhãn lớp. Vì vậy, LDA tìm hướng chiếu sao cho khả năng phân tách giữa các lớp là tốt nhất.
- Trong bài toán phân loại nhị phân với dữ liệu mất cân bằng, LDA có thể hỗ trợ mô hình nhận diện lớp thiểu số tốt hơn.

Kết quả thực nghiệm và đánh giá

Đánh giá theo Confusion Matrix

- Các mô hình nhìn chung dự đoán tốt lớp 0, sự khác biệt giữa các mô hình chủ yếu nằm ở khả năng phát hiện lớp 1.
- Mỗi ma trận nhầm lẫn có dạng:

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

Bảng: Một số ma trận nhầm lẫn tiêu biểu trên tập Test cố định

Mô hình	Dữ liệu	Tỷ lệ	Confusion Matrix		Nhận xét				
Logistic	Original	6:4		<table><tr><td>155</td><td>6</td></tr><tr><td>1</td><td>10</td></tr></table>	155	6	1	10	Chỉ bỏ sót 1/11 mẫu lớp 1
155	6								
1	10								
Logistic	LDA	6:4		<table><tr><td>157</td><td>4</td></tr><tr><td>2</td><td>9</td></tr></table>	157	4	2	9	F1-score cao, ít nhầm lớp 0 hơn
157	4								
2	9								
KNN	Original	6:4		<table><tr><td>155</td><td>6</td></tr><tr><td>6</td><td>5</td></tr></table>	155	6	6	5	Bỏ sót 6/11 mẫu lớp 1
155	6								
6	5								
SVM	Original	6:4		<table><tr><td>156</td><td>5</td></tr><tr><td>6</td><td>5</td></tr></table>	156	5	6	5	Recall thấp
156	5								
6	5								
SVM	LDA	6:4		<table><tr><td>156</td><td>5</td></tr><tr><td>2</td><td>9</td></tr></table>	156	5	2	9	LDA cải thiện phát hiện lớp 1
156	5								
2	9								

Kết quả thực nghiệm và đánh giá

Nhận xét từ Confusion Matrix

- **Logistic Regression** trên dữ liệu gốc với tỷ lệ **6:4** có ưu điểm nổi bật là chỉ bỏ sót 1 mẫu lớp 1.
- Cụ thể, mô hình phát hiện đúng 10 trên 11 mẫu thuộc lớp 1, tương ứng với $Recall = 0.9091$
- Trong bối cảnh bài toán y tế, đây là kết quả quan trọng vì giúp giảm nguy cơ bỏ sót bệnh nhân có khả năng mắc bệnh.
- **Logistic Regression kết hợp LDA** cũng cho kết quả tốt với 9 mẫu lớp 1 được phát hiện đúng và chỉ bỏ sót 2 mẫu.
- Với **KNN** và **SVM** trên dữ liệu gốc, số mẫu lớp 1 bị bỏ sót còn cao. KNN trên dữ liệu gốc bỏ sót 6/11 mẫu lớp 1, trong khi SVM trên dữ liệu gốc cũng bỏ sót 6/11 mẫu lớp 1
- Khi sử dụng **LDA**, SVM được cải thiện rõ rệt, số mẫu lớp 1 phát hiện đúng tăng lên **9/11**.

Kết quả thực nghiệm và đánh giá

Chuyển đổi bài toán phân loại sang hồi quy

- Để đánh giá khả năng mô hình hóa mức độ rủi ro thay vì chỉ phân loại nhị phân, nhóm thực hiện chuyển đổi bài toán từ phân loại sang hồi quy.
- Biến mục tiêu hồi quy được xây dựng từ xác suất dự đoán lớp dương của Logistic Regression:

$$y_i^{reg} = P(Biopsy = 1 \mid x_i)$$

- Trong đó, $y_i^{reg} \in [0, 1]$ thể hiện mức độ rủi ro của từng mẫu dữ liệu.
- Nhóm sử dụng mô hình Linear Regression và KNN Regressor để học ánh xạ từ x_i sang y_i^{reg} trên hai tập dữ liệu: dữ liệu gốc và dữ liệu sau khi giảm chiều bằng PCA.

Kết quả hồi quy sau khi chuyển đổi từ bài toán phân loại

Mô hình	Dữ liệu	MAE	MSE	RMSE	R^2
Linear Regression	Original	0.0452	0.0066	0.0812	0.8858
KNN Regressor	Original	0.0418	0.0158	0.1256	0.7271
Linear Regression	PCA (1/3)	0.0607	0.0125	0.1120	0.7830
KNN Regressor	PCA (1/3)	0.0405	0.0114	0.1068	0.8027

Kết quả thực nghiệm và đánh giá

Nhận xét kết quả hồi quy

- **Hiệu quả mô hình:** Linear Regression trên dữ liệu gốc đạt kết quả tốt nhất với $R^2 = 0.8858$ và $RMSE = 0.0812$, cho thấy mô hình xấp xỉ tốt xác suất đầu ra từ Logistic Regression.
- **Ảnh hưởng của PCA:** PCA làm giảm hiệu suất của Linear Regression do mất một phần thông tin, nhưng giúp KNN Regressor cải thiện rõ rệt:

$$R^2 : 0.7271 \rightarrow 0.8027$$

- **Kết luận:** Việc chuyển bài toán phân loại sang hồi quy là khả thi. Linear Regression phù hợp nhất trên dữ liệu gốc, trong khi PCA hỗ trợ tốt hơn cho KNN Regressor.

Kết quả thực nghiệm và đánh giá

Kết quả phân cụm K-Means với $K = 2$

- K-Means được áp dụng trên dữ liệu đầu vào sau khi bỏ qua nhãn *Biopsy*.
- Mục tiêu là khảo sát cấu trúc cụm tự nhiên của dữ liệu.

Kết quả đánh giá phân cụm K-Means với $K = 2$

Chỉ số	Giá trị
Silhouette Score	0.5502
Davies-Bouldin Index	1.7283
Adjusted Rand Index	0.0950

Phân bố nhãn Biopsy trong từng cụm

Cluster	Biopsy = 0	Biopsy = 1
Cụm 0	736	43
Cụm 1	67	12

- Tỷ lệ mắc bệnh trong từng cụm:

$$\text{Cụm 0 : } \frac{43}{736 + 43} \approx 5.52\%, \quad \text{Cụm 1 : } \frac{12}{67 + 12} \approx 15.19\%$$

Kết quả thực nghiệm và đánh giá

Nhận xét kết quả phân cụm K-Means

- **Silhouette Score** đạt 0.5502, cho thấy dữ liệu có xu hướng hình thành cụm ở mức trung bình. Tuy nhiên, **Adjusted Rand Index** chỉ đạt 0.0950, cho thấy các cụm do K-Means tạo ra không khớp rõ ràng với nhãn *Biopsy* thực tế.
- Cả hai cụm đều chứa đồng thời mẫu của hai lớp, nghĩa là K-Means chưa phân tách tốt nhóm có bệnh và không có bệnh.
- Cụm 1 có tỷ lệ mẫu dương tính cao hơn cụm 0, tuy nhiên số lượng mẫu lớp 1 vẫn bị phân tán trong cả hai cụm.
- Nguyên nhân là do K-Means là thuật toán không giám sát, chỉ dựa trên khoảng cách giữa các điểm dữ liệu và không sử dụng thông tin nhãn.
- Ngoài ra, dữ liệu bị mất cân bằng mạnh với **803 mẫu lớp 0** và chỉ **55 mẫu lớp 1**, nên K-Means có xu hướng phân cụm theo cấu trúc tổng thể thay vì tách riêng lớp bệnh.
- Vì vậy, K-Means chỉ phù hợp để khảo sát cấu trúc tiềm ẩn, không phù hợp để thay thế các mô hình phân loại có giám sát trong bài toán dự đoán *Biopsy*.

Kết quả thực nghiệm và đánh giá

Kết luận phần thực nghiệm

- Thứ nhất, dữ liệu ban đầu bị mất cân bằng mạnh. SMOTE được áp dụng trên tập *Train* để cân bằng dữ liệu huấn luyện.
- Thứ hai, Logistic Regression là mô hình có hiệu quả ổn định nhất trên tập *Test* cố định
- Thứ ba, PCA giúp giảm chiều dữ liệu và giữ lại trên 95% phương sai, nhưng không luôn cải thiện hiệu quả phân loại.
- Thứ tư, LDA có tác động tích cực đến KNN và SVM, giúp hai mô hình này cải thiện *Recall* trên lớp 1.
- Thứ năm, K-Means chưa phản ánh tốt nhãn *Biopsy*, do đó chỉ phù hợp để tham khảo cấu trúc dữ liệu.
- Tóm lại, nếu ưu tiên giảm bỏ sót bệnh nhân, nên chọn **Logistic Regression – Original** với tỷ lệ **6:4**. Nếu ưu tiên cân bằng Precision, Recall và F1-score, Logistic Regression - LDA hoặc Logistic Regression - Original/PCA ở tỷ lệ 4:1 cũng là lựa chọn tốt.

Nội dung

- 1 Tổng quan về bài toán
- 2 Tiền xử lý dữ liệu
- 3 Giảm chiều và trực quan hóa dữ liệu
- 4 Xây dựng mô hình
- 5 Kết quả thực nghiệm và đánh giá
- 6 Kết luận**

Kết luận

Tổng kết quá trình thực nghiệm

- Đề tài xây dựng quy trình ứng dụng học máy để dự đoán nguy cơ ung thư cổ tử cung dựa trên bộ dữ liệu *Cervical Cancer Risk Factors*.
- Bài toán được xác định là bài toán **phân loại nhị phân** với biến mục tiêu là *Biopsy*:
 - Lớp 0: Không có dấu hiệu ung thư cổ tử cung.
 - Lớp 1: Có dấu hiệu ung thư cổ tử cung.
- Sau tiền xử lý, dữ liệu gồm **858 mẫu**, trong đó có **803 mẫu lớp 0** và **55 mẫu lớp 1**, cho thấy dữ liệu bị mất cân bằng mạnh.
- Phương pháp **SMOTE** được áp dụng trên tập *Train* nhằm cân bằng dữ liệu huấn luyện và hỗ trợ mô hình học tốt hơn lớp thiểu số.
- Nhóm sử dụng các mô hình **KNN**, **Logistic Regression**, **SVM** và **K-Means**; đồng thời đánh giá tác động của **PCA** và **LDA**.

Kết luận

Kết quả đạt được và hướng phát triển

- **Logistic Regression** là mô hình ổn định nhất trên tập *Test* cố định.
- Kết quả tốt nhất nếu ưu tiên giảm bỏ sót bệnh nhân là **Logistic Regression – Original** với tỷ lệ **6:4**:

$$Accuracy = 0.9593, \quad Precision = 0.6250, \quad Recall = 0.9091, \quad F1 = 0.7407$$

- Mô hình phát hiện đúng **10/11 mẫu lớp 1**, chỉ bỏ sót 1 mẫu, phù hợp với mục tiêu ưu tiên *Recall* trong bài toán y tế.
- **PCA** giúp giảm chiều dữ liệu nhưng không luôn cải thiện phân loại; **LDA** thường cải thiện khả năng nhận diện lớp 1, đặc biệt với KNN và SVM.
- **K-Means** chỉ phù hợp để khảo sát cấu trúc dữ liệu, chưa thể thay thế các mô hình phân loại có giám sát.
- Hạn chế của đề tài là dữ liệu còn nhỏ, lớp 1 ít, tồn tại nhiều giá trị thiếu và chưa kiểm chứng trên nguồn dữ liệu độc lập.
- Hướng phát triển: thử nghiệm thêm Random Forest, XGBoost, LightGBM, lựa chọn đặc trưng và mở rộng dữ liệu thực tế.